

AISHA — MASTER BUILD

Master Feature Workflow, Operating Logic & Implementation Specification

STATUS: MASTER BUILD

This is the master build reference for AISHA, not a v1 release specification. It defines how the complete locked architecture operates, how data moves between modules, how features interact, and what must exist before a feature is considered implemented.

Master rule: no locked module or feature is silently removed. If implementation is staged, it remains pending rather than disappearing from the master scope.

1. MASTER PRODUCT DEFINITION

AISHA: A calm, intelligent companion that grows closer over time and feels emotionally real through a controlled software relationship model. The system combines conversation, memory, mood, relationship progression, presence, productivity, voice, 3D avatar behavior, security, cloud synchronization and long-term state management.

AISHA may express affection, care, concern or missing the user as generated behavior based on software state. These behaviors must not be represented as proof of human consciousness or biological emotion.

2. MASTER ARCHITECTURE

User → UI → Application Services → Core Intelligence → AI Integration → Output Systems → Memory/State → Security/Persistence → Cloud/Background Services

Layer	Master components
Application	Home; Chat; Voice; Day Logs; Mood; Relationship; Settings
Core Intelligence	Identity Engine; Mood Engine; Love Bond Engine; Decision Engine; Presence Engine; Conversation Engine; Safety/Rules Engine
AI Integration	Input Processing; Prompt Builder; Language Model; Response Validator
Memory	Working Memory; Day Log Manager; Recall Engine; Export Manager; Trash Manager; Recovery Manager
Avatar	Avatar Controller; Expression Engine; Animation Engine; Eye/Face; Lip Sync; Clothing/Appearance; Live Wallpaper
Productivity	Task Manager; Reminder Engine; Daily Planner; Notification Manager
Security	Authentication; Authorization; Secure Local Storage; Encryption; Key Management; Audit
Cloud	Sync; Upload Queue; Download Queue; Backup; Storage Management
Administration	Admin Dashboard; Super Admin; Recovery; Monitoring
System Services	Day Finalization; Sync; Notification; Battery; Update; Crash Recovery; Storage; Connectivity

3. MASTER INTERACTION PIPELINE

User action/input → Input Processing → Identity + Mood + Bond + Memory context → Decision Engine → Safety/Rules → Prompt Builder → Language Model → Response Validator → Output → State update → Day Log → encrypted local persistence → optional cloud sync

Feature-specific branches may add processing, but they must not bypass validation, security, state ownership or required persistence.

4. APPLICATION INTERFACE WORKFLOW

Home

Load current AISHA state → retrieve mood/bond/tasks/presence summary → render avatar → expose navigation.

Chat

Receive message → master AI pipeline → validated reply → update eligible state → avatar → Day Log.

Voice

Capture permitted audio → speech recognition → master AI pipeline → TTS → playback/lip-sync → optional encrypted audio log.

Day Logs

Select date → authorize → retrieve/decrypt → integrity check → render timeline/summary.

Mood

Read structured short-term state → display values/timeline; never expose hidden chain-of-thought.

Relationship

Read Bond Engine → display stage/progress/events allowed by product rules.

Settings

Read/write preferences → validate → secure persistence → notify affected services.

5. CORE INTELLIGENCE WORKFLOW

Identity Engine

Owns stable identity, personality constraints and persistent behavioral configuration.

Mood Engine

Processes meaningful events and updates bounded short-term values: happiness, calmness, energy, affection and curiosity.

Love Bond Engine

Evaluates long-term relationship events such as consistency, trust and shared experiences; progresses gradually through configured stages.

Decision Engine

Combines input, identity, mood, bond, memory, tasks, presence and settings to choose the next action/response intent.

Presence Engine

Handles greetings, quiet presence, return-after-absence behavior and permitted proactive actions while respecting user controls.

Conversation Engine

Coordinates context, memory, decision, generation, validation and delivery.

Safety/Rules Engine

Enforces consent, privacy, boundaries and product rules before delivery or state changes.

6. AI GENERATION WORKFLOW

Stage	Operation
Input Processing	Detect language, intent, entities and immediate context.
Context Assembly	Load identity, mood, bond, relevant memory and active task/presence context.
Decision	Select response/action intent.
Safety/Rules	Apply consent, privacy, safety and product constraints.
Prompt Builder	Create model context from approved information.

Language Model	Generate candidate natural-language output.
Response Validator	Check safety, tone, consistency, privacy leakage and memory accuracy.
Delivery	Send validated result to text, voice or approved output.
State Update	Process eligible mood, bond, task and presence changes.
Persistence	Write Day Log and encrypted local state; queue sync if enabled.

7. MEMORY & DAY LOG MASTER WORKFLOW

Event → classify importance → write current Day Log → update indexes/state → encrypt → local storage → optional sync

Recall: identify topic/date/context → retrieve relevant records → authorization/integrity check → decrypt → provide approved context to conversation pipeline. Missing information is never invented.

Day finalization: close day → summarize → finalize mood/bond timeline → integrity check → compress where useful → encrypt → local save → sync queue → start next day.

8. MOOD + RELATIONSHIP WORKFLOW

Mood is short-term. Bond is long-term. They are related but are not the same state variable.

Conversation/event → classify → Mood update → Bond eligibility check → response/presence context → avatar state → Day Log

Relationship progression is gradual and rule-controlled. It depends on sustained interaction, trust/shared experiences and configured conditions. The system does not use coercion, emotional blackmail, isolation or deliberate dependency engineering.

9. PRESENCE, TASKS & NOTIFICATIONS

Presence

Context trigger → permission/quiet-setting check → relevance evaluation → no action / quiet presence / greeting / permitted notification → execute → log.

Task Manager

Create/edit/complete → validate → save → expose status to planner/reminder/day log.

Reminder Engine

Read schedule → check time/settings → trigger or queue notification → record status.

Daily Planner

Read tasks/reminders/context → organize day → user remains in control.

Notification Manager

Receive permitted request → apply interruption rules → schedule/deliver → record result.

10. 3D AVATAR MASTER WORKFLOW

Approved Core State → Avatar Controller → Expression + Animation → Eye/Face + Body → Lip Sync when speaking → renderer/live wallpaper

Expression Engine

Maps approved mood/interaction state to neutral, happy, thoughtful, concerned and future expressions.

Animation Engine

Breathing, blinking, posture changes, looking around, walking/sitting and natural idle variation.

Eye/Face

Natural eye contact, occasional glance-away behavior, blinking and subtle facial movement.

Lip Sync

Uses speech/audio timing to synchronize mouth movement.

Clothing/Appearance

Customization is stored separately from mood/relationship state.

Live Wallpaper

Ambient avatar behavior follows battery/privacy/user settings and does not imply microphone/camera listening.

11. VOICE MASTER WORKFLOW

Permission → audio capture → speech recognition → Input Processing → master AI pipeline → validated response → TTS → playback → lip sync/avatar → optional encrypted voice log → Day Log → sync

12. SECURITY, AUTHENTICATION & AUTHORIZATION

Authentication → authorization → secure local access → encryption/key operations → feature access

Protected data path: raw data → compression where applicable → encryption → local storage and/or encrypted cloud upload → access control → retention policy.

Roles: User, Admin, Super Admin. Sensitive privileged operations must be authorized server-side; client-side role flags are never trusted as the sole security control.

Supported architecture includes offline-only mode, account mode for backup/sync, Android authentication such as Google Sign-In, and future authentication methods through the same authorization layer.

13. CLOUD, BACKUP & SYNCHRONIZATION

Local DB → encryption layer → Sync Manager → Upload Queue → cloud storage. Local source remains until successful confirmation according to policy.

Restore: encrypted backup → integrity validation → local decryption → database/index rebuild → resume.

Conflict: detect versions/timestamps → apply explicit merge/version policy → preserve data according to that rule → never silently overwrite.

Cloud retention: the master architecture retains the documented 40-day cloud retention rule. Cloud expiry must not automatically delete active local data.

14. STORAGE, EXPORT, DELETION & RECOVERY

Storage

Separate local working data, encrypted cloud backup, sync queues, protected recovery/trash and user exports.

Compression

Compress suitable data before encryption; do not expect encrypted data to compress efficiently.

Export

User request → authorization → gather permitted records → readable summary and/or encrypted archive → secure delivery.

Deletion

Remove from active timeline/normal recall → synchronize deletion state where configured → controlled recovery lifecycle.

Protected Trash

Encrypted/protected recovery area excluded from normal memory recall.

Recovery

Authorize → validate → restore → rebuild indexes → audit event.

15. ADMINISTRATION & AUDIT

Admin authentication → role authorization → permitted dashboard action → audit record

Admin functions include monitoring, authorized recovery and maintenance. Super Admin covers higher-risk security policy, key-management, role and disaster-recovery workflows.

Audit includes admin logins, authorization failures, recovery, security changes, sensitive exports and relevant security events.

16. BACKGROUND SERVICES

Service	Master operation
Day Finalization	Finalize day → summarize → validate → compress/encrypt → queue sync → create next day.
Sync Service	Process upload/download queues → retry failures → verify completion.
Notification Service	Schedule and deliver permitted notifications.
Battery Manager	Reduce background/avatar workload under power constraints.
Update Manager	Apply supported updates and database migrations while preserving compatible data.
Crash Recovery	Restore safe state after failure and prevent corrupted sessions from continuing silently.
Storage Manager	Monitor usage, thresholds and cleanup/recovery policies.
Connectivity Monitor	Detect online/offline transitions and control sync/retry behavior.

17. FAILURE-HANDLING MASTER MATRIX

Failure	Required behavior
No internet	Local features continue; cloud operations queue.
Cloud unavailable	Encrypted local state remains usable; retry later.
Upload interrupted	Keep local source; mark pending; retry.
AI service unavailable	Preserve state; use approved degraded/local path if available; do not fabricate a response.
Phone restart	Recover local DB, state and queues.
App update	Run migration and preserve compatible data.
Phone lost	Recover only from an available valid cloud backup; device-only data cannot be reconstructed without backup.
Unauthorized admin	Reject and audit.
Corrupt backup	Validate before restore; reject unsafe restore.

18. COMPLETE END-TO-END SCENARIOS

Normal text: user message → input processing → identity/mood/bond → memory → decision → rules → prompt → model → validator → reply → state update → avatar → Day Log → encryption → sync.

Voice: microphone permission → audio → speech recognition → same intelligence pipeline → TTS → playback → lip sync → avatar → optional encrypted voice record → Day Log → sync.

Return after absence: app opens → Presence Engine detects return → user settings → recent context → decision → validated greeting/presence → avatar → Day Log. No fabricated absence events.

Offline: local operation → encrypted local persistence → pending sync objects → connectivity returns → queue processing → confirmation.

Day transition: collect events → finalize mood/bond → summarize → integrity check → compress/encrypt → save → sync queue → new day.

19. SAFE ACTIVITY / THINKING LOG

The app may show operational summaries such as “checking recent memory,” “reviewing today's context,” “adjusting response tone,” “updating relationship state,” “saving today's event,” or “syncing encrypted data.” It must not expose private hidden chain-of-thought.

20. TEST & SIMULATION MASTER REQUIREMENTS

Six-month scenarios: interactive user, busy user and ignored user. Track mood, bond, memory, presence, stage transitions and long-term recall.

24-hour simulation: first login → morning → task → text → voice → mood event → bond event → avatar → offline → connectivity return → sync → day finalization.

Security simulation: failed logins, authorization failures, credential/key rotation, interrupted upload, corrupt backup, cloud outage, recovery and audit verification.

Storage simulation: text-heavy, voice-heavy, media-heavy, 40-day retention, offline queue growth and storage thresholds.

21. MASTER STATE OWNERSHIP

Each state has one authoritative owner: Identity Engine owns identity; Mood Engine owns mood; Love Bond Engine owns bond; Task Manager owns tasks; Memory/Day Log owns historical records; Avatar Controller owns rendering state; Security layer owns access/encryption; Sync Manager owns synchronization state. Other modules may read approved state but must not silently overwrite another module's authoritative state.

22. MASTER IMPLEMENTATION CONTRACT

Every module/feature is complete only when it has: Input → Validation → Processing → Output/state change → Persistence → Security → Error handling → UI/API integration → Background behavior where required → Tests → Recovery behavior.

A screen existing is not enough. The underlying engine, data path, state management, security, failure handling and integration must work.

23. MASTER EXTENSION POINTS

The master architecture reserves interfaces for controlled preference learning, communication-style adaptation, memory importance scoring, improved semantic retrieval, mood smoothing, advanced micro-expressions, avatar outfits/idle behavior, seasonal themes, smarter reminder timing, multi-device sync, MFA/additional authentication, anomaly detection, enhanced key rotation, export controls, admin reporting, prompt optimization/model selection and improved conflict resolution.

These are extension points, not reasons to remove or weaken the master foundation.

24. MASTER BUILD CHECKLIST

1. Application UI and navigation
2. Core Intelligence engines
3. AI integration and response validation
4. Memory, Day Logs and recall
5. Mood and Love Bond state
6. Presence and proactive behavior
7. Tasks, reminders and notifications
8. 3D avatar and rendering pipeline
9. Voice and lip sync
10. Authentication and authorization

11. Encryption and key management
12. Cloud backup/sync and queues
13. Storage, retention, export and recovery
14. Trash/deletion lifecycle
15. Admin and audit
16. Background services
17. Offline/failure handling
18. Simulation and automated tests
19. Migration/update safety
20. Post-launch extension interfaces

25. FINAL MASTER BUILD RULE

Nothing from the master architecture is to be silently dropped, replaced by a fake placeholder, or marked complete without its workflow and integration. If implementation is staged, its status is tracked as pending implementation while it remains part of the master build.

This document is the master operational reference for building AISHA. Future improvements extend the architecture rather than rewriting the foundation.